Session 8: Virtual Memory

---

1. What is Virtual Memory?

Virtual Memory is a memory management technique that provides the illusion of having more memory than physically available by using a combination of RAM and secondary storage (e.g., hard disk).

- **Key Features**:
    - Allows large programs to run even when physical memory is limited.
    - Provides memory isolation between processes.
    - Enables multitasking by efficiently managing memory.
- **How It Works**:
    - Divides programs into small chunks called **pages**.
    - Only the required pages are loaded into RAM, while others remain in secondary storage until needed.

Example:

- Physical RAM = 4 GB.
- Virtual memory extends this to 16 GB by using 12 GB of disk space.

---

2. Demand Paging

Demand Paging is a technique where pages are loaded into memory only when they are required, rather than loading the entire program into RAM.

- **How It Works**:
    1. A process starts execution.
    2. If the required page is not in memory, a **page fault** occurs.
    3. The operating system fetches the page from secondary storage and loads it into memory.
- **Advantages**:
    1. Reduces initial loading time.
    2. Saves memory by loading only necessary pages.

Example:

- A program requires 10 MB of memory, but only 2 MB of pages are active at a time. Demand paging loads only the 2 MB into RAM, deferring the rest.

---

3. Page Faults

A **page fault** occurs when a program tries to access a page that is not currently in RAM.

- **Steps During a Page Fault**:
    1. The operating system pauses the process.
    2. Identifies the missing page.
    3. Loads the page from secondary storage into RAM.
    4. Updates the page table.
    5. Resumes the process.
- **Types of Page Faults**:
    1. **Major Page Fault**: Page is not in RAM, and the OS must load it from disk.
    2. **Minor Page Fault**: Page is already in memory but not mapped to the process's page table.

Example:

- A program accesses a page not in memory. The OS retrieves it from the disk, causing a delay.

---

4. Page Replacement Algorithms

When the physical memory is full, the operating system must decide which page to replace to make room for a new page. This decision is made using **page replacement algorithms**.

4.1 First In, First Out (FIFO)

- Replaces the oldest page in memory.
- **Advantages**: Simple to implement.
- **Disadvantages**: May replace frequently used pages.
- **Example**:
    - Pages: [1, 2, 3, 4]
    - Access Sequence: 1 → 2 → 3 → 4 → 5
    - Replacement: Page 1 is replaced first.

4.2 Least Recently Used (LRU)

- Replaces the page that has not been used for the longest time.
- **Advantages**: Reduces the chance of replacing frequently used pages.
- **Disadvantages**: Requires tracking access times, which can be complex.
- **Example**:
    - Pages: [1, 2, 3]
    - Access Sequence: 1 → 2 → 3 → 1 → 4
    - Replacement: Page 2 is replaced (least recently used).

4.3 Optimal Page Replacement

- Replaces the page that will not be used for the longest time in the future.
- **Advantages**: Theoretically the best algorithm.
- **Disadvantages**: Requires future knowledge of page references.
- **Example**:
    - Pages: [1, 2, 3, 4]
    - Access Sequence: 1 → 2 → 3 → 1 → 4 → 2 → 3
    - Replacement: Page 4 is replaced (not needed soon).

4.4 Clock Algorithm

- A practical approximation of LRU using a "clock" mechanism.
- Uses a reference bit to decide replacement.
- **Advantages**: Simple and efficient.
- **Example**:
    - Pages have a reference bit: [1(1), 2(0), 3(0)]
    - If a page's reference bit is `0`, it is replaced.

Illustrative Example of Page Replacement Algorithms

Problem:

- Physical memory: 3 frames.
- Page Reference Sequence: [7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3].

Algorithm	Page Faults	Explanation
FIFO	9	Pages replaced in the order: 7 → 0 → 1 → 2 → 3 → 4.
LRU	8	Tracks recently used pages, reducing faults compared to FIFO.
Optimal	6	Uses future knowledge to replace pages that are not needed for a long time.

This explanation provides a clear understanding of **virtual memory**, its operation, and the algorithms used for efficient page management. Examples illustrate each concept, ensuring clarity.